

DOCUMENT DE CONCEPTION FONCTIONNELLE ET TECHNIQUE

Cahier des Charges

— Version 3.0 —

Édition consolidée. Fusionne le document v1.0 (vision produit, modules fonctionnels) et le document v2.2 (architecture, modèle économique, modèle de données) en un seul référentiel, corrige les contradictions entre les deux, et comble les trous identifiés par l'analyse croisée.

v1.0 + v2.2 → v3.0 consolidée

11 contradictions/trous corrigés – voir Partie 0.2

Web + Mobile + Super Admin

PostgreSQL · RLS

SaaS multi-tenant

Guinée → Afrique de l'Ouest

Conakry · Septembre
2026

Statut : référentiel de
développement

0.1 Sommaire & méthode

Comment ce document a été construit, et comment le lire

Ce document remplace v1.0 et v2.2 comme référence unique. Il ne les jette pas : chaque module reprend le fond de v1.0, précisé par les compléments de v2.2, corrigé où les deux se contredisaient, et complété là où l'analyse a trouvé des trous. La Partie 0.2 documente chaque changement avec sa raison — pour qu'aucune décision ne soit muette.

Partie 0 — Vision, contexte, utilisateurs, registre des décisions	ce que le produit résout, pour qui, et ce qui a changé
Partie 1 — Modules fonctionnels	20 modules, champs, statuts, version cible
Partie 2 — Rôles, permissions, gouvernance multi-tenant	qui peut faire quoi, isolation des données
Partie 3 — Architecture technique	stack, multi-tenance, asynchrone, sécurité
Partie 4 — Modèle de données	inventaire des tables, 3 tables ajoutées, corrections de schéma
Partie 5 — Modèle économique	grille tarifaire définitive, monétisation des paiements
Partie 6 — Sécurité, performance, internationalisation	ce qui protège le produit et le prépare à grandir
Partie 7 — Feuille de route	MVP → V2 → V3 → Enterprise, fusionnée et priorisée
Partie 8 — Cycle de vie : tests, environnements, livrables	comment on sait que c'est fini, et bien fait
Conclusion	ce qui prime si tout le reste doit attendre



Documents de référence associés

schema.sql reste le référentiel exécutable de la Partie 4 — en cas de divergence, le schéma fait foi. Le dossier Maquette-plateforme-web-mobile.pdf illustre à quoi ressemblent ces modules à l'écran ; il reste indicatif, pas contractuel.

0.2 Registre des décisions

Chaque écart entre v1.0, v2.2 et le schéma existant — tranché, ici, une fois

Sujet	Statut	Situation avant v3.0	Décision v3.0
Paliers d'abonnement	CORRIGÉ	v1.0 §26 : 3 paliers (Free/Pro/Business). v2.2 D1 : 4 paliers (Free/Standard/Pro/Business), déjà signalé comme incohérent par v2.2 lui-même.	4 paliers retenus (Partie 5) — c'est le schéma <code>subscription_plans</code> qui fait foi.
Moyens de paiement	NOUVEAU	v1.0 §14/§26 parle de moyens « configurables » ; le schéma <code>code payment_method</code> en simple <code>varchar</code> sur <code>payments</code> — aucune table de configuration par organisation.	Table <code>payment_methods</code> ajoutée (Partie 4.2).
Promesse de paiement	NOUVEAU	v1.0 §16 mentionne l'action « enregistrer une promesse de paiement » sur un <code>impayé</code> — aucune table ne l'implémentait.	Table <code>payment_promises</code> ajoutée (Partie 4.2), liée aux <code>Impayés</code> (Partie 1).
Tickets de support SaaS	NOUVEAU	v1.0 §27 (Super-Admin) parle de « tickets » — confondu jusqu'ici avec <code>maintenance_requests</code> (fuites d'eau), qui est un concept différent.	Table <code>support_tickets</code> ajoutée, distincte de la <code>maintenance</code> (Partie 4.2).
Compte de l'équipe Géra	CORRIGÉ	<code>users.organization_id</code> est <code>NOT NULL</code> — impossible de créer un compte pour l'équipe Géra elle-même (super-admin) sans lui inventer une fausse organisation.	Table <code>platform_staff</code> séparée, hors du périmètre RLS, avec son propre <code>réalm</code> d'authentification (Partie 4.2 & Partie 2.3).
Calendrier	CLARIFIÉ	v1.0 §18 décrit un module <code>Calendrier</code> sans qu'aucune table <code>calendar/events</code> n'existe nulle part.	Confirmé : c'est une vue agrégée , pas une table — elle interroge <code>invoices.due_date</code> , <code>contracts.end_date</code> , <code>maintenance_requests</code> , <code>property_visits</code> (Partie 1.3).
« Incidents » locataire	CLARIFIÉ	v1.0 §11 liste « incidents » séparément des « demandes de maintenance » dans l'historique, sans jamais définir la différence.	Décision : pas de table séparée pour le MVP — « incident » = synonyme de <code>maintenance_requests</code> . À rouvrir seulement si un vrai besoin de « litige » distinct émerge (ex. lié à l'état des lieux, A2).
Multi-devise	CORRIGÉ	v1.0 §39 promet l'ajout futur d'USD/EUR/CFA, mais <code>units</code> , <code>contracts</code> , <code>invoices</code> , <code>payments</code> , <code>expenses</code> n'ont pas de colonne <code>currency</code> (seules les tables SaaS en ont une).	Colonne <code>currency varchar(3) DEFAULT 'GNF'</code> ajoutée sur les 5 tables dès le MVP — migration triviale maintenant, coûteuse après coup (Partie 4.2).
Internationalisation	CLARIFIÉ	v1.0 §40 promet le multilingue ; aucune architecture <code>i18n</code> n'était nommée dans v2.2.	Chaînes externalisées dès le MVP (<code>gettext Django</code> , <code>next-intl</code> , <code>intl Flutter</code>) même si seul le français est livré (Partie 6.3).
Environnements de déploiement	CLARIFIÉ	v1.0 §55 : 4 environnements (<code>Dev/Testing/Staging/Prod</code>). v2.2 B14 n'en décrit que 2 (<code>staging/prod</code>).	3 environnements retenus : Développement (<code>local + CI éphémère</code>), <code>Staging</code> , <code>Production</code> (Partie 8.2).
Rôle Owner-viewer	CORRIGÉ	v2.2 ajoute le rôle <code>owner_viewer</code> (A3) sans mettre à jour la matrice de permissions de v1.0 §23.	Matrice de permissions réécrite avec 6 rôles (Partie 2.2).

Le problème

La gestion locative reste largement faite au cahier, à Excel, au reçu papier et sur WhatsApp : informations perdues, échéances oubliées, impayés mal suivis, aucun historique fiable, aucune visibilité consolidée sur les revenus, dépenses et interventions de maintenance.

Vision — les 11 questions auxquelles répondre en un coup d'œil

- Combien de biens possède-t-il ? Combien sont occupés ? Disponibles ?
- Qui sont ses locataires ? Qui a payé ? Qui doit encore payer ?
- Combien doit-on lui verser ? Quels sont ses revenus ? Ses dépenses ?
- Quels loyers arrivent bientôt à échéance ? Quels travaux sont en cours ?

Utilisateurs cibles

Profil	Rôle
Propriétaire particulier	1 à quelques biens, gestion directe
Gestionnaire immobilier	Gère pour le compte d'un ou plusieurs propriétaires
Agence immobilière	Plusieurs propriétaires, plusieurs biens
Employé / Agent	Opérations autorisées (paiements, locataires)
Locataire	Espace personnel optionnel (V3, portail — A9/§43)

Types de biens gérés

Résidentiel	Commercial	Autre
Maison, appartement, chambre, studio, villa	Magasin, boutique, bureau, entrepôt, local commercial	Terrain, parking, type configurable

Objectif

Transformer une gestion locative dispersée en gestion centralisée, automatisée, sécurisée, accessible depuis n'importe quel appareil — sans en faire, au MVP, un système trop vaste pour être livré (voir Partie 7).

Module	Contenu clé	Statuts / cycle	Renvoi BDD	Cible
Tableau de bord	Propriétés, locaux occupés/dispo, revenus du mois, impayés, dépenses, bénéfice estimé, contrats expirant. Graphiques : revenus, impayés, occupation, dépenses, paiements par moyen.	—	Requêtes agrégées sur invoices/payments/units, cache Redis	MVP
Propriétés	Nom, type, adresse, ville, quartier, description, photo, propriétaire, statut.	Actif / archivé	properties	MVP
Locaux / unités	Numéro, type, étage, superficie, description, loyer, caution, propriété associée.	Disponible → Réservé → Occupé → Maintenance → Hors service	units	MVP
Locataires	Identité, contact, profession, pièce d'identité, contact d'urgence. Historique : contrats, paiements, impayés, anciennes locations, demandes de maintenance (« incidents » = synonyme, voir 0.2).	—	tenants	MVP
Garants A1	Nom, téléphone, adresse, profession, pièce d'identité, lien avec le locataire, montant garanti, document signé. Alerte si contrat « garant exigé » sans garant.	—	guarantors	MVP
Contrats	Locataire + local + conditions : dates, loyer, caution, fréquence, jour d'échéance, conditions. Renouvellement, résiliation, suspension, PDF.	Brouillon → Actif → Suspendu → Résilié → Expiré	contracts	MVP
Révision des loyers A11	Augmentation périodique (taux fixe, indice, décision manuelle) avec conservation de l'historique des montants successifs.	—	rent_revisions	V2
État des lieux A2	Entrée/sortie, état par pièce, photos horodatées, signature. Comparaison auto entrée/sortie → proposition de répartition de la caution, reliée aux Dépenses.	—	inspections, inspection_items	V2

01 Modules fonctionnels — Argent

v1.0 §13–16, §26 (corrigé) + v2.2 A4–A5

Module	Contenu clé	Statuts / cycle	Renvoi BDD	Cible
Facturation	Génération auto des factures de loyer par période active. Ex. Facture #FAC-2026-00125 — 2 000 000 GNF, échéance 05/10/2026.	En attente → Partielle → Payée → En retard → Annulée	invoices	MVP
Palements	Locataire, contrat, local, montant, date, période, moyen, référence, utilisateur enregistrant.	—	payments	MVP
Moyens de paiement NOUVEAU	Espèces, Orange Money, MTN MoMo, virement, carte, et moyens additionnels configurables par organisation.	Actif / inactif	payment_methods	MVP
Reçus de paiement	Numéro, propriétaire, locataire, local, montant, période, date, moyen, référence, solde restant. Actions : PDF, imprimer, partager, envoyer.	—	Généré depuis payments	MVP
Pénalités de retard A5	Calcul financier configurable (fixe, %, dégressif, plafond), remplace la simple classification qualitative du retard.	—	late_fee_rules	MVP
Impayés	Détection auto des factures en retard : locataire, local, montant dû + pénalité, jours de retard, historique. Actions : relance, paiement partiel, promesse de paiement.	Retard léger / moyen / important	invoices.status	MVP
Promesses de paiement NOUVEAU	Un locataire en retard promet un montant à une date. Suivi tenue/rompue — alimente le score de fiabilité (V3).	En attente → Tenue → Rompue	payment_promises	V2
Commission d'agence A4	Taux fixe/variable par bien ou propriétaire, base loyer brut/net, fréquence de reversement. Sortie : bordereau de versement.	—	agency_commission_rules, owner_payouts	MVP
Portail propriétaire A3	Vue lecture seule (rôle owner_viewer) : loyers perçus, commission, dépenses imputées, net à verser, relevés PDF mensuels.	—	owner_payouts	MVP
Dépenses	Catégories (plomberie, électricité, peinture, nettoyage, réparation, gardiennage, taxes...), propriété/local, montant, justificatif.	—	expenses	V2

Module	Contenu clé	Statuts / cycle	Renvoi BDD	Cible
Notifications	Moteur J-7 / J-3 / J / J+1 / J+7 configurable. Canaux : interne, email, SMS, WhatsApp.	En file → Envoyée → Échec	notification_rules, notifications	MVP
Calendrier CLARIFIÉ	Vue agrégée — échéances, contrats, renouvellements, interventions, rendez-vous. Filtrable par propriété/local/locataire/type.	—	Vue calculée invoices + contracts + maintenance_requests + property_visits	MVP
Prospection & visites A6	Cycle bien vacant → visite → candidature → sélection → contrat. Conversion en locataire en un clic.	Nouveau → Visite → Négociation → Converti / Perdu	leads, property_visits	V2
Signature électronique A7	Capture de signature manuscrite, horodatage, hash du document. Intégration tierce possible (Yousign, DocuSign).	—	esignatures	V3
Portail locataire	Contrat, loyer, factures, paiements, reçus, dettes, échéances, demandes de maintenance. Signalement de problème, envoi de document.	—	Vue restreinte sur tenants	V3
Assistant WhatsApp A9	« Quel est le solde de Mamadou ? », « Paiement de 500 000 GNF reçu pour M-003 » — entrée conversationnelle en plus des notifications sortantes.	—	Orchestré via payments/invoices	V3
Maintenance	Ticket : propriété, local, locataire, problème, priorité, photos, prestataire, coût.	Nouveau → Assigné → En cours → Résolu → Fermé	maintenance_requests	V2
Prestataires A10	Annuaire (plombier, électricien, peintre...) avec historique, coût moyen, délai moyen, notation — remplace le champ libre « responsable ».	—	vendors	V2
Import / migration A8	Assistant CSV/Excel : mapping de colonnes, détection de doublons, validation avant import — propriétés, locaux, locataires, contrats.	—	Écrit dans les tables cibles	V2

01 Modules fonctionnels — Gouvernance & pilotage

v1.0 §21, §24, §27, §30

Module	Contenu clé	Statuts / cycle	Renvoi BDD	Cible
Documents	Contrats, pièces d'identité, reçus, factures, justificatifs, photos — jamais stockés en base, stockage objet.	—	documents	MVP
Journal d'activité	Toute opération sensible tracée — qui, quoi, quand. Écriture seule, jamais modifiable ni supprimable.	—	activity_logs	MVP
Rapports (organisation)	Financier (revenus, paiements, impayés, dépenses, bénéfice), immobilier (occupation), locataire (historique, dettes). Export PDF / Excel-CSV.	—	Requêtes agrégées	V2
Conformité légale locale A12	Préavis, plafond légal de caution, motifs de résiliation — à valider avec un juriste guinéen, jamais codés en dur.	—	Paramètres configurables par organisation	MVP
Support SaaS NOUVEAU	Une organisation cliente ouvre un ticket à l'équipe Géra — distinct des tickets de maintenance des locataires.	Ouvert → En cours → Résolu → Fermé	support_tickets	V2
Administration plateforme	Vue super-admin : organisations, abonnements, paiements SaaS, statistiques MRR/ARR/churn, plans, limites, suspension de comptes.	—	organizations, subscriptions, platform_invoices	MVP
Mode hors-ligne étendu A13	Au-delà des paiements (déjà prévu) : consultation locataires/contrats et création de tickets de maintenance hors-ligne.	—	Base locale SQLite + file de synchronisation	V3
Vue multi-agence A14	Un propriétaire confiant ses biens à plusieurs agences obtient une vue consolidée cross-organisation.	—	Requête cross-org contrôlée	ENTERPRISE

Deux couches de KPI, à ne jamais confondre

Les KPI d'une
organisation

(occupation, revenus, impayés, bénéfice — v1.0 §45) mesurent la santé d'une agence ou d'un propriétaire. Les KPI de la
plateforme

(MRR, ARR, churn, ARPU, CAC — v2.2 D4) mesurent la santé de Géra elle-même. Le tableau de bord d'Ibrahima affiche les premiers ; le poste super-admin affiche les seconds — jamais les deux mélangés.

02 Rôles, permissions, gouvernance multi-tenant

v1.0 §22–25 corrigé (rôle owner_viewer intégré)

Matrice de permissions — 6 rôles

Fonction	Owner	Admin	Manager	Agent	Owner-viewer	Tenant
Propriétés	✓	✓	✓	👁	👁 (siennes)	—
Locataires	✓	✓	✓	✓	—	👁 (soi)
Paielements	✓	✓	✓	✓	👁	👁 (soi)
Dépenses	✓	✓	✓	✗	👁	—
Rapports financiers	✓	✓	✓	✗	👁 (siens)	—
Utilisateurs	✓	✓	✗	✗	—	—
Paramètres	✓	✓	✗	✗	—	—

✓ accès complet · 👁 lecture seule · ✗ aucun accès. Le rôle **Owner-viewer** (A3) est strictement scindé du rôle Owner opérationnel : un même propriétaire peut être Owner d'une organisation qu'il gère et Owner-viewer sur les biens confiés à une agence tierce.

Isolation multi-tenant

Chaque compte professionnel (organization) dispose de son propre espace. Aucune donnée du client A n'est accessible au client B — garanti par Row Level Security PostgreSQL (Partie 3), pas seulement par le code applicatif.

Client A → Propriétés A1, A2, A3

Client B → Propriétés B1, B2

L'équipe Géra n'est pas un client

CORRIGÉ Les comptes de l'équipe Géra (support, opérations) ne peuvent pas vivre dans users, qui exige un organization_id. Ils vivent dans une table séparée platform_staff (Partie 4.2), avec un rôle PostgreSQL distinct — jamais BYPASSRLS par défaut, activé seulement pour les comptes techniques de support, action tracée dans activity_logs.

Couche	Choix retenu
Structure	Monolithe modulaire Django (apps découplées par domaine) — pas de microservices au démarrage. Premier candidat à l'extraction : les notifications.
Frontend web	Next.js / React + TypeScript, Tailwind CSS
Mobile	Flutter — un seul code base iOS/Android, hors-ligne via SQLite (drift)
Backend API	Django + Django REST Framework, Python 3.12
Base de données	PostgreSQL 16 — Row Level Security native
Multi-tenance	Schéma partagé + organization_id + RLS — décision non négociable pour le MVP
Cache / files	Redis (cache, verrous distribués) + Celery (asynchrone, planifié)
Stockage fichiers	S3-compatible, un préfixe par organisation, URLs signées
Paielements mobiles	API Orange Money / MTN MoMo — idempotence obligatoire sur les webhooks, réconciliation quotidienne
Notifications	Firebase (push), Africa's Talking/Twilio (SMS), WhatsApp Business Cloud API, SendGrid/Mailgun (email)
Observabilité	Sentry, Prometheus + Grafana, Loki
CI/CD	GitHub Actions + Docker, Terraform pour l'infrastructure
Authentification	JWT (access court + refresh rotatif) + TOTP 2FA. Comptes platform_staff sur un realm séparé.



Défense en profondeur

RBAC applicatif (rôles, Partie 2) + RLS PostgreSQL (organisation) : même une requête Django mal filtrée ne peut pas exposer les données d'une autre organisation — la base l'interdit elle-même. Comportement « fail closed » vérifié : si le contexte n'est jamais fixé, aucune ligne n'est renvoyée.

Domaine	Tables	Rôle
Tenancy & utilisateurs	organizations, users, organization_members	Frontière multi-tenant, comptes de connexion, accès multi-organisation
Équipe plateforme NOUVEAU	platform_staff	Comptes internes Géra — hors périmètre organisation/RLS
Patrimoine	owners, properties, units	Propriétaires, propriétés, locaux
Location	tenants, leads, property_visits, contracts, guarantors, rent_revisions, inspections, inspection_items, esignatures	Cycle complet locataire → contrat → révisions → état des lieux
Argent — loyers	invoices, payments, payment_methods, payment_promises, late_fee_rules	payment_methods et payment_promises ajoutées
Argent — agence	agency_commission_rules, owner_payouts	Commission & versements propriétaires
Exploitation	vendors, maintenance_requests, expenses	Prestataires, tickets, dépenses
Gouvernance	documents, notification_rules, notifications, activity_logs, support_tickets	support_tickets ajoutée, distincte de la maintenance
Modèle SaaS	subscription_plans, subscriptions, platform_invoices, usage_counters, payment_transactions	Abonnement, facturation SaaS, quotas, traçabilité mobile money

Contraintes d'intégrité (v1.0 §29 — confirmées)

- *Un paiement doit être associé à un contrat valide (via une facture).*
- *Un contrat doit être associé à un locataire et à un local existants.*
- *Un local occupé doit avoir un contrat actif — vérifié par cohérence applicative + trigger.*
- *Une facture ne peut être soldée au-delà de son montant sans gestion explicite du surplus (avoir / crédit).*
- *Les suppressions importantes (propriétés, locaux, locataires) passent par suppression logique (deleted_at), jamais un DELETE direct.*

payment_methods — moyens de paiement configurables

Colonne	Type	Note
id	uuid	Clé primaire
organization_id	uuid	NULL = méthode globale par défaut (espèces, Orange Money, MTN MoMo)
code, label	varchar	Identifiant technique et libellé affiché
requires_reference	boolean	Exige un numéro de transaction (ex. Mobile Money)
is_active, sort_order	boolean, smallint	Activable/désactivable par organisation, ordre d'affichage

payment_promises — promesses de paiement (v1.0 §16)

Colonne	Type	Note
id, organization_id, invoice_id, contract_id	uuid	Rattachée à la facture en retard
promised_amount, promised_date	numeric, date	Ce qui a été promis, et pour quand
status	varchar	pending / kept / broken
created_by, created_at, kept_at	uuid, timestamptz	Traçabilité

support_tickets — assistance Géra ↔ organisation cliente

Colonne	Type	Note
id, organization_id	uuid	L'organisation qui contacte le support
subject, message	varchar, text	Objet de la demande
priority, status	varchar	open / in_progress / resolved / closed
created_by	uuid → users	Utilisateur de l'organisation cliente
assigned_to	uuid → platform_staff	Membre de l'équipe Géra en charge

Correction — colonnes de devise (v1.0 §39)

Ajouter currency varchar(3) NOT NULL DEFAULT 'GNF' sur : units (rent_amount), contracts (rent_amount), invoices (amount), payments (amount), expenses (amount). Migration triviale au MVP — coûteuse en production avec des millions de lignes existantes.

Palier	Cible	Limites	Fonctionnalités incluses
Free	Petit propriétaire, découverte	1 propriété, 3 locaux max	Suivi de paiement basique, reçu simple
Standard	Propriétaire avec quelques biens	Locaux étendus	Reçus PDF, rapports de base, pénalités de retard
Pro	Multi-biens / petit gestionnaire	Locaux illimités	Employés, état des lieux, signature électronique, portail propriétaire
Business	Agences immobilières	Multi-gestionnaires	Commission automatisée, API, comptabilité, bot WhatsApp

4 paliers — remplace définitivement les 3 paliers de v1.0 §26 (voir 0.2). Prix à valider par entretiens de disposition à payer, pas figés à ce stade.

Monétisation des paiements — mode retenu

Locataire — paiement direct → Propriétaire / Agence ← frais de service fixes Géra

Mode **facilitateur** retenu pour démarrer : frais fixes par transaction, jamais un pourcentage prélevé sur le flux — un prélèvement en % qualifierait Géra d'intermédiaire financier, potentiellement soumis à agrément de la Banque Centrale de la République de Guinée. Le mode agrégateur (commission automatique) reste un pari réglementaire de phase Enterprise.

Indicateurs à suivre dès les premiers clients payants

MRR, ARR, churn mensuel, ARPU (particuliers vs agences, séparément), CAC et ratio LTV/CAC, taux de conversion Free → payant.

■ Sécurité

- *HTTPS, hash sécurisé des mots de passe, 2FA TOTP*
- *Contrôle d'accès par rôle (RBAC) + par organisation (RLS)*
- *Protection CSRF, limitation de débit, validation des entrées*
- *Protection injections SQL & XSS, gestion sécurisée des fichiers*
- *Chiffrement applicatif des champs sensibles (pièces d'identité)*
- *Logs de sécurité, analyse automatique des dépendances (Dependabot, bandit)*

■ Performance & sauvegardes

- *Index ciblés (organization_id, status, due_date)*
- *select_related/prefetch_related dès le MVP, pas « au besoin »*
- *Pagination systématique, traitement asynchrone (Celery)*
- *Sauvegardes automatiques + tests de restauration réguliers*
- *Répliques de lecture PostgreSQL si la charge des rapports le justifie*

■ Internationalisation & multi-devise CLARIFIÉ

Français au lancement, anglais et devises supplémentaires (USD, EUR, CFA) prévus pour l'expansion Afrique de l'Ouest. Décision : externaliser les chaînes d'interface dès le MVP (gettext côté Django, next-intl côté Next.js, package intl côté Flutter) même si une seule langue est livrée — et porter la colonne currency sur les tables monétaires dès le MVP (Partie 4.2), pour ne jamais avoir à retoucher un montant existant en production.

⚖️ **Rappel — Conformité légale locale (A12)**

Préavis, plafond de caution, motifs de résiliation : valeurs par défaut à valider avec un juriste guinéen avant la mise en production, et toujours configurables par organisation plutôt que codées en dur.

MVP

- Authentification, propriétés, locaux, locataires, contrats
- Facturation, paiements, reçus, moyens de paiement configurables
- Impayés (détection + pénalités)
- Garants, portail propriétaire + commission d'agence
- Tableau de bord (revenus, occupation, impayés)
- Décision RLS + schéma de données appliqué tel quel
- Conformité légale cadrée (paramétrable)

V2

- Facturation automatique récurrente, notifications J-7...J+7
- Dépenses, rapports, export PDF/Excel
- Maintenance + annuaire prestataires
- Utilisateurs & permissions complètes
- État des lieux, révision des loyers, prospection/visites
- Import Excel/CSV, promesses de paiement, support SaaS
- Intégration mobile money (webhooks + réconciliation)

V3 / Enterprise

- Application mobile complète, portail locataire
- Paiements en ligne (mode facilitateur), bot WhatsApp
- Signature électronique, mode hors-ligne étendu
- Multi-devises, API publique, automatisations avancées
- **ENTERPRISE** Multi-agences, comptabilité, audit complet, intégration bancaire

✓ Critères de réussite du MVP (v1.0 §50)

Un utilisateur crée son compte → une propriété → ses locaux → ses locataires → un contrat → le système génère les échéances → un paiement est enregistré → un reçu est généré → les impayés sont détectés → les statistiques se mettent à jour — le tout avec des données isolées entre organisations, des rôles fonctionnels, et une sauvegarde automatique en place.

Tests

Type	Portée
Unitaires	Fonctions et services métier (calcul de pénalité, de commission, de solde)
API	Authentification, propriétés, contrats, paiements, factures
Intégration	Communication entre modules (paiement → facture → reçu → notification)
Sécurité	Permissions par rôle, isolation inter-organisations, validation des entrées, upload de fichiers
Utilisateur (E2E)	« Créer une propriété → un local → un locataire → un contrat → un paiement → générer le reçu »

Environnements CLARIFIÉ

Développement (local + CI) → Staging → Production

3 environnements retenus, pas 4 : le « Testing » de v1.0 §55 est absorbé par la CI (tests automatiques à chaque pull request), pas un environnement persistant séparé.

Livrables

Application Web · API Backend documentée (OpenAPI/Swagger) · Base de données (schema.sql) · Application mobile selon la phase · Interface d'administration (super-admin) · Documentation technique & utilisateur · Système de sauvegarde · Monitoring · Tests automatisés · Procédure de déploiement.

Ce projet n'est pas une application CRUD : c'est une plateforme SaaS multi-utilisateur, multi-propriété, multi-organisation, conçue pour évoluer sans réécriture — de la Guinée vers l'Afrique de l'Ouest.



La priorité absolue du MVP

Savoir quels biens sont loués, qui doit payer, combien a été payé, combien reste à payer, et combien le propriétaire gagne. Une fois ce socle maîtrisé — schéma de données appliqué, RLS en place, portail propriétaire et commission d'agence livrés — les fonctionnalités avancées s'ajoutent progressivement sans compromettre la stabilité.

■ Ce que ce document a changé, en une phrase

v1.0 posait la vision et le détail fonctionnel ; v2.2 posait l'architecture, l'économie et le schéma ; **v3.0 les rend cohérents entre eux** — 4 paliers au lieu de 3, 3 tables ajoutées (payment_methods, payment_promises, support_tickets) et une quatrième pour l'équipe elle-même (platform_staff), le calendrier et les « incidents » clarifiés, la devise et l'internationalisation anticipées avant qu'elles ne coûtent une migration.